



Implementación de Métodos Computacionales

**Introducción al paradigma de
programación Funcional**

Paradigma Funcional

- Basado en la evaluación (o aplicación) de funciones.
- Mayor nivel de abstracción al programar.
- Planteamiento de la solución a un problema, basado en un “QUE” y no en “COMO”.

Evaluación de funciones

- Todo programa es una función, que dadas ciertas entradas, genera ciertos resultados.

Definición de la función f

$$f(x, y) = 2x - 5y$$

Diagram illustrating the definition of a function f . The expression $f(x, y) = 2x - 5y$ is shown. The parameters x and y are labeled "Parámetros" (Parameters) with arrows pointing to them. The expression $2x - 5y$ is circled and labeled "Cuerpo de la función" (Body of the function).

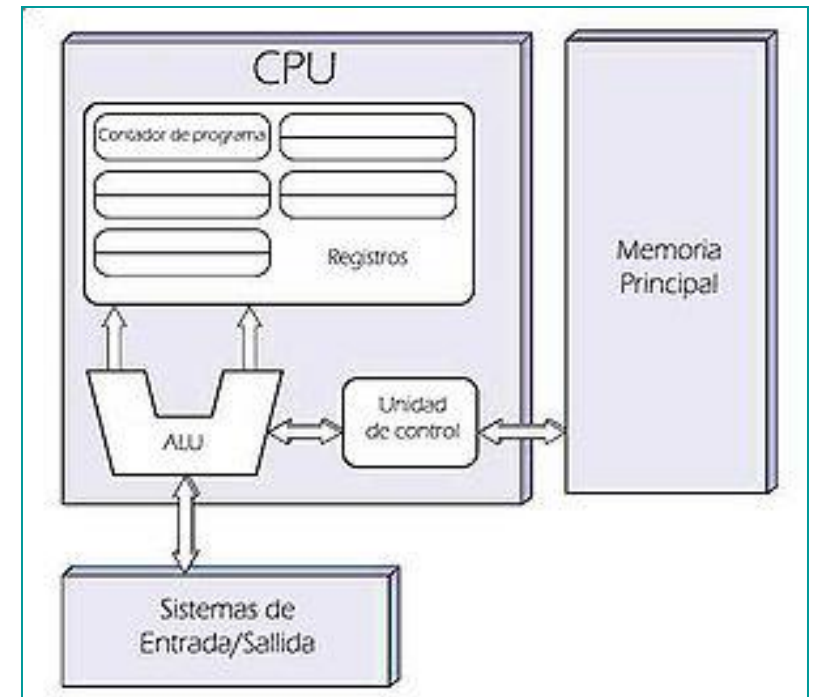
Aplicación o evaluación de la función f

$$f(7, 2) \Rightarrow 4$$

Diagram illustrating the application or evaluation of the function f . The expression $f(7, 2) \Rightarrow 4$ is shown. The arguments 7 and 2 are labeled "Argumentos" (Arguments) with arrows pointing to them. The result 4 is circled and labeled "Resultado" (Result).

Mayor nivel de abstracción

- Alejado del modelo de la máquina de Von Neumann.
- No construye soluciones mediante modificaciones de estado.
- Resuelve los problemas principalmente mediante la combinación de funciones.
- Modularidad obligada: cada función es un módulo.



Diferencias entre Paradigmas

Estilo Imperativo

```
Function Factorial(n : int): int;  
Var fact, j : int;  
Begin  
    fact = 1;  
    For j = 2 to n do  
        fact = fact * j;  
    Return (fact);  
End;
```

Estilo Funcional

```
Function Factorial(n : int): int;  
Begin  
    If n == 0 then  
        Return (1)  
    else  
        Return (n * Factorial(n-1));  
End;
```

Ventaja Consecuente: TRANSPARENCIA REFERENCIAL

Un módulo con el estilo funcional (o función):

- ❑ NO depende de variables o valores globales.
- ❑ No utiliza variables locales.
- ❑ No realiza asignaciones.
- ❑ Controla su ejecución con decisiones y recursividad.

Fácil lectura, mantenimiento, paralelización y comprobación.

Fundamentación teórica: CALCULO LAMBDA

- Propuesto por Alonso Church en 1941.
- Notación especial para definir funciones sin nombre.
- Las funciones pueden ser argumentos de otras funciones, o bien, resultado de otras funciones.

$$\lambda x . \lambda y . x + y$$

Lenguajes Funcionales

- **LISP**

- John McCarthy, 1958
 - FranzLisp
 - CommonLisp

- **APL** (Array programming language)

- Kenneth Iverson, 1964

- **LOGO** (Lisp without parentheses)

- Daniel G. Bobrow, Wally Feurzeig & Seymour Papert, 1967

- **SCHEME** (Dialecto de Lisp)

- Guy Steels & Gerald Sussman, 1975

- **FP** (Function Programming)

- John Backus, 1977

- **ML** (MetaLanguage)

- Robin Milner, 1973

- **MIRANDA**

- David Turner, 1985

Principales aplicaciones

- Inteligencia Artificial
- Problemas científicos
- Sistemas para comprobación matemática
- Manipulación simbólica
- Lógica
- Programación concurrente
- ...

Otras Características de los lenguajes funcionales

- **Funciones como elementos de primer orden.**
 - Se pueden utilizar como argumentos.
 - Se pueden generar como resultado.
 - Pueden ser manipuladas como los datos.
- **Administración de la memoria con bindings dinámicos.**
 - Los datos NO están restringidos en su capacidad.
 - La única estructura de datos que se provee es la **LISTA**.
- **La evaluación de funciones está sujeta a las condiciones de ejecución (lazy evaluation).**
- **Normalmente utilizan un INTERPRETE como traductor.**

Desventajas

- **Menor eficiencia en la ejecución de programas.**
 - Interpretación
 - Traducción de un nivel mayor.
 - Acceso a estructuras de datos: Listas vs. Arreglos
- **Conflicto con el manejo de operaciones de entrada/salida.**
 - La solución han sido los lenguajes híbridos o no puros.
- **“Lesión cerebral” causada por la forma tradicional (procedural/imperativa) de programar.**